

L4 Safety Island for Autoware

Software design, Part 1 — node allocation

國立臺灣大學



Prof. Chi-Sheng (Daniel) Shih

National Taiwan University • Reference Design WG • Version 20260918 • draft for review

Revision history

Major revisions only. The version is a date-code, YYYYMMDD, and all four artefacts of this deliverable — design document, this deck, the review edition, the discussion document — carry the same one.

Version	Issued	Status and scope
20260918	2026-09-18	Major revision. B1 recast as an ODD switch negotiation — the island declares the current domain unsatisfiable, the HPC picks one it can meet, the island verifies it; B1 gains both exits and the safety timeout T_{odd} . v_{hold} defined as a decaying ramp rather than a constant. B3 gains an island-detected refuge when H8 offers none, bounded by T_{refuge} . Open decisions 17 and 18 added
20260912	2026-09-12	Editorial revision. §4 of the design gains a five-phase figure series elaborating the node allocation. No normative change
20260903	2026-09-03	Major revision. B4 declared as the fourth behaviour; the optional HPC object list admitted; B3 and B4 both recognised as terminal; hazard lamps raised at entry; the abort ladder drawn; open decisions 15 and 16 added
20260827	distributed 2026-08-28	First WG review draft. Three declared behaviours B1 – B3 , the node allocation, the interface contract, and fourteen open decisions

Takeaway. Everything new since 20260827 is marked in marker yellow. A reviewer of that set can read the marking alone and see the whole change.

Agenda

1. Where we are
2. What the island must do
3. How the allocation was decided
4. The allocation
5. Island sensing (D7)
6. Pull-over MRM
7. ODD degradation
8. Interfaces and deployment
9. Minimal configuration

Reading the identifiers

The prefix says what *kind* of thing an identifier is — and what it commits the design to.

Symbol	Names	The property it carries	In this deck
D <i>n</i>	WG <i>decision</i>	Settled before Part 1 began. An <i>input</i> to the allocation, not something this deck may reopen	D1–D7
B <i>n</i>	Declared <i>behaviour</i>	Normative, and the only normative thing here. What the island does, seen from outside. Every allocation below must trace to one	B1–B4
R <i>n</i>	Partitioning <i>rule</i>	A test, applied per node — which side of the link it runs on. R5 decides most of them	R1–R8
T <i>n</i>	Criticality <i>tier</i>	An integrity class and a dependency constraint: the ASIL a node is built to, and — the part that bites — what it may depend on	T0 T1 T1s T2 T3
P <i>n</i>	Execution <i>partition</i>	A scheduling unit, not an integrity class. One executor at a fixed priority band; the numbering is the order the island <i>sheds</i> them under overload	P0–P4
I <i>n</i>	<i>Imported node</i>	Exists upstream today. Migrated to the island; the argument for it is <i>provenance</i> , so it moves unmodified or not at all	18 on the target path
N <i>n</i>	<i>New island node</i>	Has to be written. Named <code>si_*</code> ; nothing upstream provides it, so each is a cost to justify against a behaviour	19 new
H <i>n</i>	<i>New HPC node</i>	Has to be written — by the other team. What the island needs the HPC to publish: where this design asks something of someone not in the room	9 new

A chip's colour is its tier

T0 core · **T1** supervision · **T1s** sensing · **T2** nominal · **T3** non-real-time · **HPC** other domain. The chip says the tier, not the prefix: **I6** and **I10** are imported packages at different integrity, **P2** a *partition* at T1s. **T is how well a node is built; P is when it runs.** Every chip is a link.

D1 is not “decision 1”

Two unrelated sequences share the letter. **D** *n*, no hyphen, is a WG input and is *closed*. **Open decision** *n* — **D-*nn*** in the discussion document — is still to be settled; those are the last three slides.

Where we are

What the WG has already
decided, and what is already
running on the bench.



Inputs already fixed by the WG

Decision

- D1 The L4 Safety Island is a **standalone, safety-certified ECU** — not an on-SoC partition
 - D2 HPC hosts perception / planning / VLM; the island hosts **status monitoring, the control module, and MRM**
 - D3 HPC ↔ island is **TSN / Vehicle Ethernet**; island → actuators is CAN FD / FlexRay
 - D4 Island middleware is **nano-ros** — zenoh-pico core, ROS-message compatible, RTOS-portable
 - D5 Island OS and nodes must be **real-time compliant**: predictable execution time, memory safe
 - D6 Every safety-relevant payload carries **end-to-end protection** — CRC, counter, data ID, age; gPTP provides the time base. **AUTOSAR is out of scope: Autoware in-house design**
 - D7 **Low-resolution, high-robustness sensors connect directly to the island** — 2D lidar, forward camera, ultrasonic, radar — so the MRM nodes execute without the HPC
-

Takeaway. Part 1 answers one question: **which ROS nodes execute where, and which new nodes each side needs.**

Two engineering facts, not assumptions

Already ported

autoware-safety-island (branch nano-ros) runs the vendored Autoware **MPC lateral + PID longitudinal** controllers as standalone firmware on Zephyr / FreeRTOS.

- ▶ Subscribes: trajectory, odometry, steering report, acceleration, operation-mode state
- ▶ Publishes: `control_cmd`
- ▶ Wire-compatible with *unmodified* ROS 2 Humble Autoware
- ▶ Closed loop at ≥ 19 Hz against a 10 Hz trajectory
- ▶ 250-point trajectory bound, KEEP_LAST depth 1 on every input

Already measured

Six-node RT-evaluation island from the Autoware vehicle-interface subsystem, FreeRTOS / zenoh-pico.

- ▶ Control cadence held *through a full Linux outage*
- ▶ Mean period **9.96 ms**
- ▶ 500 ms watchdog tripped in **0.49–0.67 s**
- ▶ MRM ramped the vehicle to standstill

Takeaway. The allocation is built so the existing island grows into the target island **by addition, not by rewrite.**

Four behaviours, declared before
anything is allocated — everything
after this exists to deliver them.

What the island must do



THE
AUTWARE
FOUNDATION

Four declared behaviours

This is the **normative** part of Part 1. Every allocation decision that follows must be traceable to one of these four.

ID	Behaviour	Statement
B1	Degrade the ADS when the ODD cannot be held	Require the ADS to move to an operating domain it <i>can</i> satisfy — progressively constraining it, and demanding it constrain itself, until it does — before any authority is taken
B2	Keep the vehicle in its lane after HPC loss	When the HPC is gone and B1 therefore <i>cannot</i> be used, take authority and hold the lane for a bounded window, at a decaying speed the island can itself justify, while staging the exit manoeuvre — pending the HPC's return, or escalation
B3	Pull over if the ODD continues to fail	When degradation does not restore the envelope within a declared deadline, bring the vehicle out of the running lane to a standstill in a refuge — nominated by the HPC where one is offered, detected by the island's own sensors where one is not
B4	Emergency stop when the island is blind	When the sensing that B2 and B3 <i>both</i> depend on is gone, decelerate to a standstill where the vehicle already is — no lane keeping, no refuge, no lateral manoeuvre

Takeaway. A one-way ladder with two exceptions: **B1** needs a live HPC, **B2** replaces it when there isn't one, **B3** ends the drive — and only **B2** → **B1** and **B1** → ADS_ACTIVE climb back up. **B4** is not a rung of that ladder: it is the **floor underneath it**. Two behaviours stop the vehicle, not one: **B3** places the stop, **B4** cannot. And **B1** is the only one with an **upward** exit.

Each behaviour, defined

	B1 degrade	B2 keep lane	B3 pull over
Trigger	ODD margin falls; sensor degradation; HPC declares its own ODD exit	HPC unavailable (N2) while the lane is still observable — i.e. B1 has no executor	B1's T_{odd} expired with the HPC alive; or B2 's T_{hold} expired — entered at v_{entry}
Authority	Island constrains, HPC executes. No authority taken	Island steers and brakes. Nominal follower deselected, no HPC in the loop	Island steers, brakes, signals — the only behaviour where it initiates lateral displacement
Response	notify → demand an ODD switch → restrict (v_{max}) → request MRM. The cap is held <i>underneath</i> the switch, not instead of it	Track the corridor (map → perceived lanes → heading tube) on the v_{hold} ramp , not at a constant speed, hazards on	Search both refuge sources → hazards on at entry → signal → displace → decelerate → secure
Deadline	T_{ack} 2 s to declare · T_{odd} 10 s to be in it · 3 s comply · 10 s begin · 60 s complete	Transfer ≤ 70 ms; hold at most T_{hold} — finite, never open-ended ; at T_{hold} the vehicle is already at v_{entry}	T_{refuge} 5 s to find a refuge; then 10 s begin · 60 s complete
Observable	/si/out/odd_verdict, /si/out/constraints. MrmState stays NORMAL	N3 mode LANE_KEEP_HOLD, corridor source, deviation, hold budget left	MrmState = MRM_OPERATING, behaviour PULL_OVER
On failure	Non-compliance is a fault; a domain declared but not entered is a <i>false declaration</i> — a SOTIF event at N11	Corridor lost → in-lane stop; AEB or actuator fault → emergency stop	Abort ladder → in-lane stop (B2 authority) → emergency stop. Search <i>inconclusive</i> at T_{refuge} → B4
Exits	Up: a domain that passes both acceptance tests → normal L4 operation, in a narrower domain. Down: T_{odd} expires → blackout → B4 , HPC lost → B2 , otherwise → B3	Up: HPC re-qualified over a confirmation window → B1 at CONSTRAIN. Down: T_{hold} expires → B3	Standstill in refuge → SAFE_STOP. Terminal — exactly as B4 is
Min. sensor rung	C — needs the radar/lidar divergence and camera contrast tests	A — 2D lidar + IMU. Must survive on the smallest set	B — adds rear-corner radar above ~ 30 km/h

B1 — the ODD switch: what the island demands, and what it verifies

new in 20260918

Read as a constraint ladder, **B1** is a demand that the ADS *slow down*. That is only its **enforcement**. Its object is an **operating domain**.

#	Who	What happens
1	island N19	Declares the current domain unsatisfiable : <code>odd_satisfiable = false</code> + the measured envelope + the dimension that failed. Names no target domain
2	HPC	Selects a domain it can meet. This is planning, and it happens <i>entirely</i> on the HPC
3	HPC H4	Declares the new <code>active_odd</code> and its bounds — within $T_{ack} = 2\text{ s}$
4	island N19	Verifies the declaration : those bounds must lie inside the island's own measured envelope
5	island N2	Verifies the behaviour : measured speed, acceleration and curvature converge on them — within $T_{odd} = 10\text{ s}$
6	island N3	Accepts, and releases the cap to ALLOW

The cap does not come off for the declaration. It comes off for the behaviour. Between 1 and 6 the vehicle is held at v_{max} whatever the HPC has declared, so a false declaration buys the HPC no speed — only time, and only T_{odd} of it.

The asymmetry is the point

The island holds no catalogue of the HPC's operating domains, no model of which L4 behaviours each admits, and no means of choosing between them — all three are planning, which R4 and R5 keep off the island.

But a *measurement* is enough to **reject** an answer without being able to **produce** one.

Steps 4 and 5 are separate on purpose

Step 4 catches an HPC reporting a domain it *could not* be in. Step 5 catches one reporting a domain it *is not* in — the more dangerous failure, because the declaration is plausible. Passing 4 and failing 5 is a **SOTIF event**: §6.4's confidently-wrong case, on a second channel.

B1 — the only behaviour with an *upward exit* new in 20260918

B3 and **B4** end the drive cycle; **B2** holds one open. **B1** is the only behaviour that returns the vehicle to **normal L4 operation** — in a narrower domain than the one it started in, driven by the HPC, with the island back to supervising. That narrowing is **B1**'s *outcome*, not a residual fault.

At T_{odd} expiry	Takes over	Why
Sensing blackout (N17)	B4	Capability, not situation. Neither B2 nor B3 has a precondition left
HPC unavailable (N2)	B2	The executor of every B1 rung is gone, and the corridor is still observable
HPC alive, non-compliant or out of satisfiable domains	B3	The situation has not been recovered and will not be. The drive ends in a refuge

Honest exhaustion is not non-compliance — it does not reach this table at all

“None of my domains is satisfiable” is a **compliant** answer, not a timeout. It routes straight to **Request MRM** — an HPC-executed pull-over, with the map, the traffic law and the full planner available to it, which is strictly better than the island-executed one under **B3**. T_{odd} bounds **silence and non-compliance**, not honesty.

Takeaway. The successor is chosen by what the island still **has** — never by what the HPC did wrong, because the island cannot always tell a non-compliant HPC from a dead one, and does not need to. The tests run **in that order**: capability loss, then executor loss, then situation loss — the order in which each removes the preconditions of the options below it.

B4 — the floor: emergency stop when blind

new in 20260903

Trigger	N17 declares sensing blackout — every exteroceptive source invalid at once. Equivalently: B2 has no corridor <i>and</i> B3 no verifiable refuge. Also N9 actuator fault, or I19 AEB
Precondition	None, deliberately. A fallback whose entry can be blocked is not a fallback
Authority	Brakes, and declines to steer. No corridor, so nothing to steer against; steering held under N7's rate limit, relaxing to straight
Response	Hazards on at entry → I16 commands directly → a_{emg} ramp → I15 secures. No trajectory is planned
Deadline	Transfer ≤ 70 ms. a_{emg} is open decision 15
Observable	MrmState = MRM_OPERATING, behaviour EMERGENCY_STOP — the one value the existing enum carries
Exits	SAFE_STOP only — no edge back up. Terminal even if sensing returns mid-stop
Min. sensor rung	0 — none. Proprioceptive channels only

Why stopping in the lane is accepted

The argument is *comparative*. The only other things a blind island can do are continue on stale evidence, or attempt a manoeuvre it cannot verify — both worse, and both failing **unboundedly**. A stop is bounded by the vehicle's own kinetic energy.

The residual risk is real: it belongs in the HARA as a **named accepted risk**.

B4 is not “AEB fired”

I19 is a *reflex* against one detected obstacle, presupposing a working forward sensor. B4 is a *state*: the island no longer knows what is ahead at all. AEB escalates *into* B4; the reverse is impossible.

Four episodes, two edges back up — and a floor

Precedence

- ▶ **B1** is the **first response**, and it is cooperative: the HPC stays in control. It therefore *requires a working HPC*.
- ▶ **B2** **replaces B1 when that requirement fails**. Same intent — buy time rather than commit — but the authority is inverted: the island steers. Admissible only while the lane is observable; bounded by T_{hold} .
- ▶ **B3** is **terminal for a worsening situation**: B1 exhausted its deadlines, or B2's hold expired with the HPC still gone.
- ▶ **Two behaviours end the drive, not one**. B3 is a **placed** stop — refuge verified; B4 is an **unplaced** one, promising a standstill and nothing about *where*. Both terminal, both to SAFE_STOP, two HARA entries.
- ▶ **One-way except two edges back up**: **B2** → **B1** — a re-qualified HPC gets authority back at CONSTRAIN; and **B1** → ADS_ACTIVE — the switched ODD passed both acceptance tests and held, so the HPC drives normally in the narrower domain.
- ▶ **B4** is **underneath all of them**, not after them. B1–B3 escalate on a worsening situation; B4 on a loss of *capability*, which can go at any rung. **Every state has an edge into it; its only edge out is SAFE_STOP**.

Containment supervision is not B2

The continuous **N5** check *while the HPC steers* is a validate function — its output is a veto and a fault, not a behaviour. **B2** is what the island does when it must produce the steering itself.

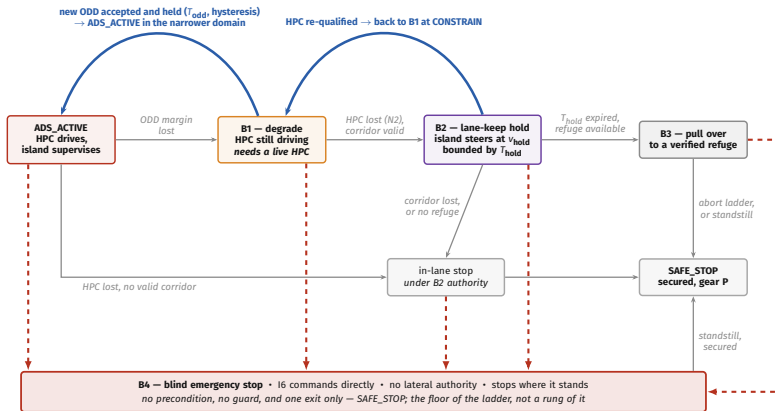
B3 deliberately leaves the lane

A pull-over leaves the running lane *on purpose*. The containment check **N5** must be **suspended under authorisation** for the duration and re-armed against the refuge boundary instead.

An unsuppressed containment check would abort every pull-over it was asked to protect.

Authorisation comes from **N3** and nothing else, bounded by **N7**, recorded by **N11**.

The ladder, drawn



ADS_ACTIVE, B1, B2, B3 and the in-lane stop each transit to B4 when all sensor and planning information is lost — sensing blackout (N17), and equally on actuator fault (N9) or AEB (I19)

Takeaway. Two edges climb: **B2** → **B1** when the HPC is re-qualified, and **B1** → ADS_ACTIVE when the switched ODD is verified and held. **B4** is the **floor**, not a fourth rung: every state above has an edge into it, and its only edge out is to SAFE_STOP — never back up the ladder.

Behaviour-to-node traceability

A node that serves no declared behaviour does not belong on the island

Detect	Decide	Act
B1 N19 ODD · N17 health · N16 world model	N10 envelope · N3 · N8	N1/H1 constraint egress → HPC re-plans
B2 N2 HPC supervisor · N13 scan · N14 vision · H3 corridor	N3 hold budget + hand-back · N5 corridor · I8 · N10	I13 pure_pursuit + N7 → I1 → I16; I3 for hand-back
B3 N16 free space · N6 rear radar · H8 refuge	N18 · I7 · N3	N7 → I13 → I1 → I16; I5 secures
B4 N17 blackout · N16 all flags false · N9 · I19	N3 (EMERGENCY_STOP, no exit) · N8 · I7	I6 → I1 → I16; I5 secures. No exteroceptive input

Takeaway. HPC loss does not commit the vehicle to a terminal manoeuvre — it commits it to a *bounded* one. When it does become terminal, it is **B3** or **B4**, and both raise the hazard lamps at entry.

How the allocation was decided

Eight rules, one of them
decisive: what must still
work after the HPC is gone.



Eight partitioning rules

Rule	Statement	Consequence
R1 Survivability	If the function must still work <i>after the HPC is gone</i> , it runs on the island	Forces the whole final-command path onto the island
R2 No orphan inputs	An island node must not depend on an HPC-only topic without a defined safe behaviour when it goes silent	Forces island-local state estimation and fault sources
R3 Monitor independence	A monitor must not share a failure mode with what it monitors	HPC liveness supervision is island-resident
R4 Bounded resources	Static memory, bounded WCET, no dynamic discovery	Excludes map-dependent and learning-based nodes from the ASIL tiers
R5 Narrow interface	Typed, fixed-rate, rate-limited, E2E-protected	Keeps HD map, point clouds, images, object lists off the link
R6 Single actuation authority	Exactly one component owns the final command, and it is on the island	The gate is the arbitration point and the ASIL-D anchor
R7 Diversity over duplication	Prefer a <i>different</i> sensing channel over a copy of the HPC's	A separate low-resolution sensor set (D7)
R8 Certifiable critical set	The ASIL-D set must be small enough to argue about	Two-tier controller

Takeaway. R1 is the decisive one. Everything else falls out of asking what must survive the HPC.

Criticality tiers — and the staleness rule

Tier	Integrity	Contents	Cadence
T0	ASIL-D	Command gate, state machine, watchdog, emergency-stop MRM, actuator TX, E2E gateway, island odometry	100 Hz
T1	ASIL-B(D)	Control validator, fallback controller, safety envelope, corridor monitor, actuator supervisor	50 Hz
T1s	ASIL-B(D)	Island sensor ingest, safety world model, emergency braking, near-field collision detection	20–50 Hz
T2	QM, monitored	MPC lateral + PID longitudinal follower, raw vehicle command converter, shift decider	33–50 Hz
T3	QM	Event recorder, UDS / DTC diagnostics, telemetry egress	1–10 Hz

ASIL decomposition by monitoring

The MPC and its OSQP solver **never need to be ASIL-D**: T0 bounds every command it produces and can replace it within one control period.

The staleness rule for sensing

T0 never blocks on T1s. Stale or invalid world state → most conservative assumption, blind deceleration MRM. *Perception improves the fallback; it is never a precondition for it.*

Reminder — what QM and ASIL-A... D actually mean

Level	What it demands	HW architectural metrics	Where it appears in this design
QM	No ISO 26262 safety requirement — developed under a normal quality system	—	T2 MPC + PID follower (<i>monitored</i>) · T3 recorder, UDS, telemetry
ASIL A	Lowest integrity. Full 26262 lifecycle, lightest verification and confirmation measures	not specified	Not used — nothing on the island lands here
ASIL B	Systematic rigour on requirements, design and test; review independence begins to bite	$SPFM \geq 90\% \cdot LFM \geq 60\% \cdot PMHF < 100 \text{ FIT}$	T1 validator, envelope, corridor, actuator supervisor · T1s sensing, AEB
ASIL C	As B, with tighter metrics and more independent verification	$SPFM \geq 97\% \cdot LFM \geq 80\% \cdot PMHF < 100 \text{ FIT}$	Not used — the decomposition below skips D to B
ASIL D	Highest. Strictest process, independent confirmation, demonstrated <i>freedom from interference</i>	$SPFM \geq 99\% \cdot LFM \geq 90\% \cdot PMHF < 10 \text{ FIT}$	T0 command gate, state machine, MRM, E2E gateway, island odometry

SPFM = single-point fault metric · LFM = latent fault metric · PMHF = probabilistic metric for random hardware failures · 1 FIT = 1 failure per 10^9 h

Where the letter comes from

The HARA assigns an ASIL to each *hazardous event* from **Severity** (S0–S3) × **Exposure** (E0–E4) × **Controllability** (C0–C3); **D** is the corner case S3 + E4 + C3 — life-threatening, likely encountered, not driver-controllable. The ASIL attaches to the **safety goal**, never to a box: components inherit it *by allocation*, which is exactly what this deck is doing.

Reading ASIL-B(D)

ASIL decomposition, ISO 26262-9: one ASIL-D requirement may be split across two **sufficiently independent** elements, each built to **B** rigour while still carrying a **D**-derived requirement — written B(D). The parenthesis is a debt, not a discount: lose freedom from interference and it collapses back to D.

Takeaway. A decomposition is only as strong as the independence argument underneath it.

The same levels, in examples

Level	A familiar vehicle function	Why it lands there	Where it appears in this design
QM	Infotainment screen freezes · seat-memory recall · cabin ambient lighting	S0 — no injury path at all	T3 event recorder, UDS export, telemetry. The T2 follower is QM only because T0 bounds it
ASIL A	Rear position lamps fail after dark	S1 · E4 · C3	Nothing — no island function has a <i>light-injury</i> worst case
ASIL B	Brake lamps or low beam fail	S2 · E4 · C2	T1 I8 validator, N10 envelope, N5 corridor · T1s N16 world model, I19 AEB — all B(D)
ASIL C	Partial, unintended loss of steering assist at speed — still steerable, with effort	S3 · E4 · C2	Nothing — the decomposition goes straight from D to B(D) + B(D)
ASIL D	Unintended airbag deployment · loss of braking · unintended steering to lock	S3 · E4 · C3	T0 I1 gate, N3 state machine, N2 supervisor, I6 emergency stop, N1 E2E, N4 odometry

S/E/C values are *illustrative* — the real ones come from the vehicle manufacturer's HARA, not from the standard. Anchor point: $S3 + E4 + C3 = D$, and every step down of one parameter drops one level.

Why an L4 island is nearly all D

In L2, part of the controllability argument is carried by the driver, so **C2** is defensible and an S3 hazard lands at **C**. Take the driver out and the same hazard is **C3** — nobody is left to catch it — and it lands at **D**.

The island's ASIL profile is not conservatism. It is the arithmetic of having no fallback-ready user.

One box, two letters

An ASIL belongs to a *hazard*, not to a component, so one node can sit at two levels at once. N16 is B(D) for making the MRM obstacle-aware — but “a stale N16 must never stall the MRM” is a **D** requirement, and it is discharged in **T0** by the staleness rule, not in N16.

Takeaway. Every letter is a property of a *hazard*, never of a box — and with no driver left to catch it, the same hazard earns a bigger letter.

The allocation

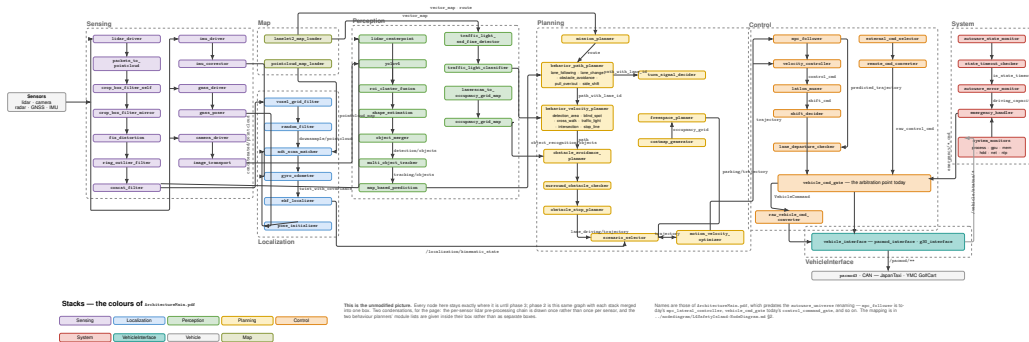
Eighteen Autoware nodes move.
The interesting part is the
three that were not obvious.



Phase 1 of 5 — Autoware today, every node

Phase 1 — Autoware today, every node

The published Autoware node graph ([autoware_node_graph.png](#)), redrawn here in the same visual language as phases 2–5 so that the five read as one progression. **Nothing on this page is changed by the Safety Island** — this is the picture the rest of the series modifies. Colour is the Autoware stack, from the original's own legend

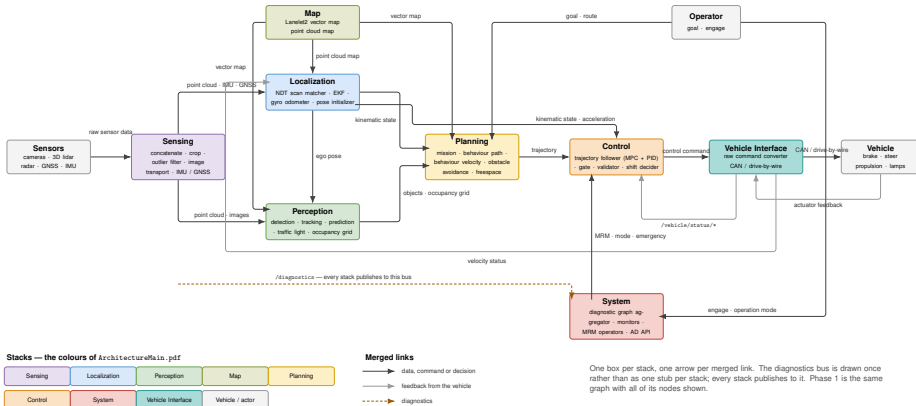


The starting point — nothing on this slide is changed by the design. It is the picture the next four phases modify.

Phase 2 of 5 — the same graph, one box per stack

Phase 2 — Autware today, one box per stack

Every node of phase 1 merged into the stack it belongs to, and every link merged with it.
This is the picture the Safety Island changes; phase 3 shows the same boxes after the change.

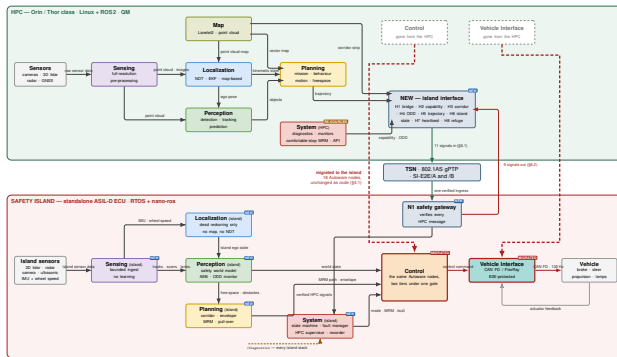


The detail folded away — every node of phase 1 merged into its stack, and every link merged with it.

Phase 3 of 5 — the same boxes, after the island

Phase 3 — the same picture, after the Safety Island

Same boxes and same colours as phase 2. The Control stack and the Vehicle Interface move bodily to the island — dashed red, from the ghost they leave behind. The island grows reduced versions of the other stacks (blue tag), the HPC gains an interface block, and one narrow link joins the two. Node-level detail is phase 4.



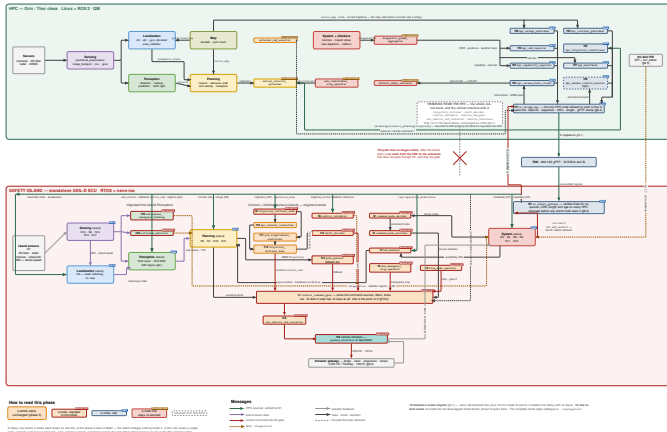
The island is not a monitor bolted on the side: it runs a reduced Autoware. Its localization is one mode, its planning is four bounded map-free ones — and its control stack is not a reimplementation at all, it is Autoware's, relocated. Phase 4 opens what changed to node level.

The change itself — Control and the Vehicle Interface migrate bodily; the island grows reduced versions of the other stacks; one narrow link joins the two.

Phase 4 of 5 — what changed, at node level

Phase 4 — what the Safety Island changes, at node level

A **satellite** zoom, between phase 2 (one box per stack) and phase 3 (the island, node-by-node).
On the HPC only the nodes that change are opened up — what was removed, what is re-acquired, and the new Veriflex block. Every stack that the island leaves alone stays as one box, exactly as in phase 2. On the island only the nodes migrated from the HPC are opened up. The island's own new stacks stay as one box each; phase 3 opens them.

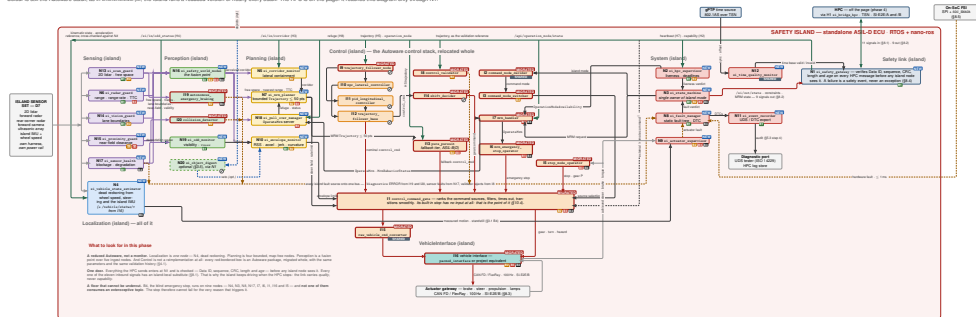


The delta, opened up — on the HPC only what was removed, re-sourced or newly designed; on the island only what migrated. Everything else stays at the phase-3 level.

Phase 5 of 5 — the Safety Island alone

Phase 5 — the Safety Island alone, every node

The last zoom. All 37 island nodes — 18 migrated Autoware ECU nodes and 19 new N-nodes — plus the optional N20.
Colour is still the Autoware stack, as in [Autoware stack](#). The island runs a reduced version of nearly every stack. The HPC is off the page: it reaches this diagram only through N1.



Stacks — the colours of [Autoware stack](#)



Where each node came from



Behaviours each node enables (S1-S10)



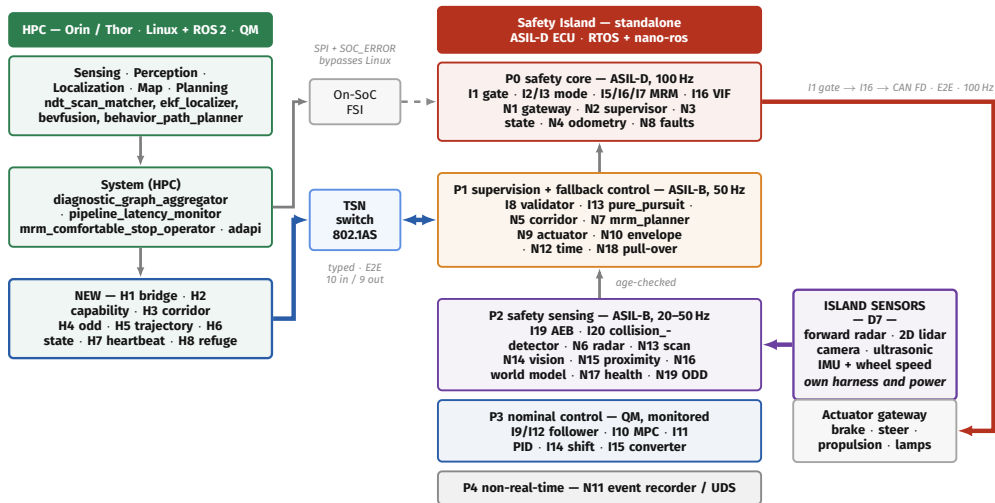
Messages



Every edge on this page, with the design document number it is taken from, is validated on "validated.ecu", which shows the same 37 nodes coloured by extension position in time.

What the island then contains — all 37 nodes: the 18 migrated Autoware nodes and the 19 new ones. The HPC reaches this picture only through N1.

Node allocation across the two domains



Two sensor sets, two harnesses, two power rails — so the only paths that survive HPC loss originate inside the island boundary.

Autware nodes on the island — authority, mode, MRM

#	Status	Package	Function on the island
I1	○	autware_control_command_gate	The arbiter. Selects among command sources, filters abnormal values, generates the built-in stop on source timeout
I2		autware_command_mode_decider	Decides which source holds authority. Plugin logic replaced by a static, table-driven decider
I3		autware_command_mode_switcher	Applies the decided mode; drives transitions
I4		autware_command_mode_types	Shared type and constant library; header-only, shared with the HPC
I5		autware_stop_mode_operator	Stop commands, optional auto-park. Does not depend on localization — reads vehicle status only
I6	○	autware_mrm_emergency_stop_operator	Emergency-stop MRM; publishes control commands directly
I7		autware_mrm_handler	Selects the MRM behaviour. Must be island-resident: its reason to act is HPC loss
I16	○	vehicle interface	CAN FD / FlexRay to the actuator gateway. Vehicle-specific, E2E-protected
I17		autware_operation_mode_transition_manager	<i>Legacy path only</i> — superseded by I2/I3
I18	○	autware_vehicle_cmd_gate	<i>Legacy gate</i> — alternative to I1. Do not deploy both

✓ vendored and running in autwarefoundation/autware-safety-island · ○ demonstrated on the RT-eval island only, ASI adoption pending · blank = not started.

Checked against the upstream repository, 2026-08.

The upstream ASI repository vendors **no** gate, mode, MRM or vehicle-interface package — its whole surface is the trajectory follower (next slide). The ○ marks come from the separate RT-eval island, which runs a command gate, an MRM operator and the 100 Hz actuator path (FreeRTOS · zenoh-pico). Whether that gate becomes I1 or I18 on the target island is **open decision 1** — hence the mark on both.

Takeaway. Not one node on this slide exists in the ASI repository today — this is phase 1 in its entirety. The target set separates *decide* / *switch* / *gate* into three nodes, exactly the decomposition a safety argument needs.

Autware nodes on the island – control, sensing, vehicle

#	Status	Package	Function on the island
I8		autoware_control_validator	Checks the control command against trajectory and vehicle limits
I9	✓	autoware_trajectory_follower_node	Host node for the two controller plugins
I10	✓	autoware_mpc_lateral_controller	Nominal lateral control. OSQP iteration cap + time budget still required
I11	✓	autoware_pid_longitudinal_controller	Nominal longitudinal control
I12	✓	autoware_trajectory_follower_base	Controller plugin base
I13		autoware_pure_pursuit	Fallback lateral control — closed-form, bounded time. New to the island
I14		autoware_shift_decider	Derives the gear command
I15		autoware_raw_vehicle_cmd_converter	Generic accel/steer to vehicle-specific pedal/torque. Must sit <i>downstream of the gate</i>
I19		autoware_autonomous_emergency_braking	Island AEB — moved from the HPC by D7. Point cloud + IMU ego path
I20		autoware_collision_detector	Near-field guard — moved from the HPC by D7. Point-cloud-only, ~5m footprint

✓ vendored and running in autwarefoundation/autoware-safety-island · ○ demonstrated on the RT-eval island only, ASI adoption pending · blank = not started.

Checked against the upstream repository, 2026-08.

Those four nodes do not travel alone. The repository also vendors `autoware_msgs`, `autoware_osqp_interface`, `autoware_universe_utils`, `autoware_motion_utils`, `autoware_interpolation` and `autoware_vehicle_info_utils` — six support libraries, not nodes, but the real width of the port. Pinned to **Autware 2025.02 / Universe 0.40.0**, CycloneDDS 0.11.x, Zephyr 3.6.0; four runtime targets (`zephyr-fvp`, `zephyr-s32z`, `freertos-posix`, `freertos-s32z2`). Interface: **5 subscriptions** — trajectory, odometry, steering report, acceleration, operation mode — **and one publication**, `control_cmd`.

Takeaway. 18 Autware nodes on the target path; 4 of them are running today — the vendored MPC + PID follower, unmodified, closed loop against stock Autware. The other 14 are additions to that image, not rewrites of it.

What stays on the HPC — and why

Whole stacks, HPC only: sensing, perception, localization, map, planning. They fail R4 (unbounded memory, learning-based, map-sized state) and R5 (their traffic would flood the link).

Package	Reason
autoware_spheric_collision_detector	Object-list based. Its point-cloud siblings moved to the island
autoware_obstacle_collision_checker, autoware_predicted_path_checker autoware_lane_departure_checker	Depend on predicted paths and map Requires the Lanelet2 vector map. Island equivalent is N5 , dual-sourced from H3 and N14
autoware_external_cmd_selector, autoware_joy_controller	Remote and manual sources. They reach the island as <i>one</i> E2E-protected command source into the gate
autoware_mrm_comfortable_stop_operator	Publishes a VelocityLimit to the planner — it cannot function without planning
autoware_diagnostic_graph_aggregator, autoware_hazard_status_converter	Large config-driven fault trees. Island runs its own static fault manager N8
autoware_system_monitor and the eight other monitors	They monitor HPC resources. Under R3 they are <i>evidence sources</i> , not the safety monitor
autoware_default_adapi_universe	API and orchestration surface

Takeaway. Consequence to handle: the island's controllers subscribe to /localization/kinematic_state and /localization/acceleration, **both HPC-produced**. Resolved by **N4**.

Design move 1 — a two-tier controller

D2 puts “the control module” on the island. Taken literally that places an iterative QP solver inside the safety-critical ECU, and the certification argument then has to cover OSQP’s worst-case execution time. **That argument is expensive and fragile.**

	Nominal tier T2	Fallback tier T1
Lateral	autoware_mpc_lateral_controller	autoware_pure_pursuit
Longitudinal	autoware_pid_longitudinal_controller	Fixed deceleration profile N7
Integrity	QM, monitored	ASIL-B(D)
Time budget	Soft — may be dropped	Hard — always meets deadline
Switch trigger	MPC budget overrun · validator reject · solver non-convergence · tracking error above bound	

The behaviour is already live

The existing port commands an emergency stop on excessive tracking error. This formalises that escalation and inserts a *graceful* intermediate step instead of going straight to a stop.

Design move 2 — two Autoware nodes can migrate

Both were assumed to need perception object lists. Their actual interfaces say otherwise, and D7 removes the remaining obstacle.

	autoware_autonomous_emergency_braking	autoware_collision_detector
Perception objects	Optional — use_predicted_object_data=false	Optional — use_dynamic_object=false
Point cloud	use_pointcloud_data=true	use_pointcloud=true
Ego path source	IMU kinematic prediction, or the MPC predicted trajectory	none; current footprint only
Map dependency	none	none
Output	ERROR on /diagnostics	ERROR on /diagnostics
Island sensor	2D lidar scan → PointCloud2; radar tracks	2D lidar + ultrasonic → PointCloud2

AEB's IMU path comes from **N4**, its point cloud from **N13**, and — usefully — its optional MPC path from **I10**, *which is already on the island*.

Takeaway. The island's emergency braking is not a bespoke reimplementaion but **the same Autoware node, same tuning parameters, same validation history** — running on a diverse sensor channel.

Design move 3 — MRM splits into two classes

Graded reaction	Implemented by	HPC alive?
1 · Notify and request self-recovery 2 · Restrict speed, enlarge margins	Island → HPC constraint / veto; HPC re-plans mrm_comfortable_stop_operator (HPC) + island envelope	Yes Yes
3a · Obstacle-aware in-lane stop	Island: fallback tier + N7 along the perceived corridor	No
3a' · Pull-over	Island: N18 to an HPC-nominated refuge	No
3b · Emergency braking on imminent collision	Island: I19 on island sensors	No
3c · Emergency stop (B4)	Island: I6 — blind, no steering, no world model needed	No
3d · Last resort	Island: gate built-in stop I1	No
4 · Audit	Island N11 + HPC log store	No

Takeaway. Steps 1–2 are cooperative degradation; 3a–3d are island-autonomous. **The island must never depend on step 2 succeeding before it can reach step 3 — and every rung above 3c has a direct edge into it,** because a guarded floor is not a floor.

Island sensing (D7)

Low resolution, high robustness
— and a second view of the
road that fails differently.



The island sensor set

Selection principle: **robustness and boundedness over resolution**. Every sensor must degrade gracefully, emit a small fixed-size payload, and be processable without a neural network.

Sensor	Link	Data class	Rate	What it gives the MRM
Forward radar	CAN FD	Object tracks: range, range-rate, azimuth	20 Hz	Longitudinal range and closing speed. Works in rain, fog, darkness, low sun
2D lidar	100BASE-T1	Planar scan, ~1080 points	10–40 Hz	Free-space distance, in-path occupancy, lateral clearance. Deterministic, no learning
Forward camera	GMSL	Lane boundaries + free-space edge	10–20 Hz	Lane geometry when the map corridor is gone
Ultrasonic	LIN / CAN FD	Near-field distances, 8–12 channels	10–20 Hz	Standstill, creep, the final metres of a pull-over
IMU + wheel speed	direct / CAN FD	Vehicle dynamics	100 Hz	Required by N4 ; also supplies AEB's IMU path

Two wiring rules

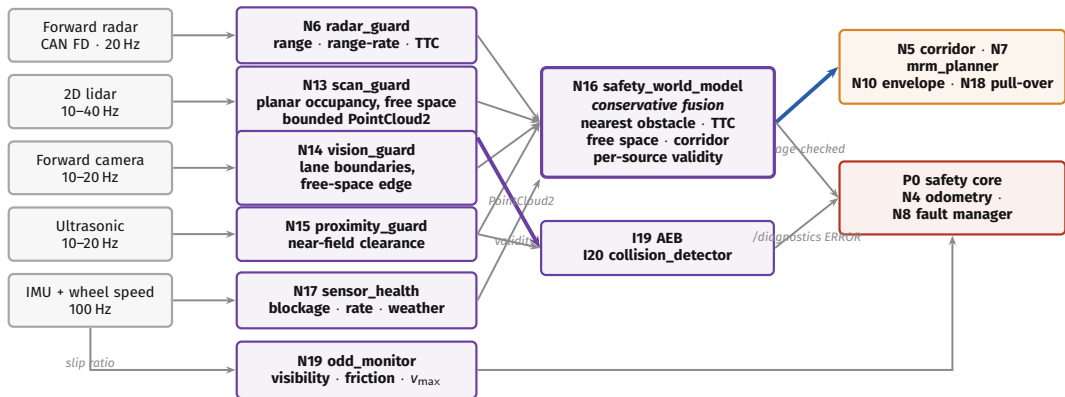
- 1. Independent harness and power.** Island sensors must not share a rail, a bundle or a switch with the HPC's set — *a common-cause failure that blinds the HPC must not blind the island*.
- 2. One-way data flow.** Sensors feed the island. No island sensing node may ever depend on an HPC-produced topic.

Five new sensing nodes

#	Node	Purpose	Rate
N13	si_scan_guard	2D lidar ingest. Fixed-size planar occupancy; free-space distance and nearest in-path return. Publishes a bounded PointCloud2 for I19/I20 . Classical, no learning, static memory	10–40 Hz
N14	si_vision_guard	Forward-camera ingest. Lane boundary polylines and free-space edge; feeds N5 as the fallback corridor source	10–20 Hz
N15	si_proximity_guard	Ultrasonic ingest. Near-field clearance per sector; gates creep, standstill release, the final metres of a pull-over	10–20 Hz
N16	si_safety_world_model	The fusion point. Combines N6, N13, N14, N15 into one bounded safety world state. <i>Conservative fusion</i> : a hazard from any source is taken as present; a corridor is trusted only when sources agree	20–50 Hz
N17	si_sensor_health	Per-sensor availability, frame rate, blockage, degradation — dirt, spray, fog, low sun, radar interference. Sets each source's validity flag for N16	5–10 Hz

Takeaway. Without D7 the island could execute a *blind* MRM. With D7 it executes an **obstacle-aware** one — brake harder for a detected obstacle, stop short of it, keep lane on perceived boundaries after the map corridor expires.

The island sensing chain



Two paths out of **N13**: a bounded PointCloud2 straight to the migrated Autoware nodes — which is what lets them run unmodified — and the fused world state for the supervision tier. The safety core reads **N16** only through an age-checked snapshot.

Pull-over MRM

The HPC nominates refuges. The island verifies one, executes it, and can abandon it at every step.



The island cannot decide *where* to pull over

Choosing a legal stopping place means knowing about intersections, crosswalks, bus stops, bridges, tunnels and local traffic law. That is map and planner work, and it stays on the HPC.

The split

The HPC continuously nominates refuges. The island verifies and executes one.

... but under **B3** the HPC is often gone **new in 20260918** — next slide.

The refuge contract — **H8**

side	LEFT / RIGHT
entry_s, exit_s	Longitudinal window, as arc length
lateral_offset_m	Target displacement from lane centre
refuge_width_m	Usable width, for clearance
surface_class	PAVED / GRAVEL / UNKNOWN
legality	LEGAL / EMERGENCY_ONLY / PROHIBITED
validity_distance_m	How far ahead it remains valid
fallback_behavior	Normally IN_LANE_STOP

The one hard guarantee

At any moment the buffered refuge set must cover **at least the current worst-case stopping distance** plus the island's fault-reaction budget.

When the HPC cannot honour that — a tunnel, a bridge, a barriered section — it must **say so explicitly** rather than fall silent.

Takeaway. Consequence: **the island always knows, before any failure, whether a pull-over is available.** **N18** maintains a continuous pull_over_available flag — fallback capability becomes an *observable*.

When the HPC offers no refuge: the island finds one

new in 20260918

B3 is reached from **B2** exactly when the HPC has been **lost** — and H8's buffered refuge set ages out like every other buffered input. An island with no second source makes the refuge contract a **single point of failure for the one manoeuvre it exists to enable**.

	H8 nominated refuge	N18 island-detected refuge
Physically free	verified by N16 , before use	verified by N16 , before use
Legal	checked against map and traffic law by the HPC	unknown — the island holds neither
legality	LEGAL_STOP or EMERGENCY_ONLY, as nominated	EMERGENCY_ONLY, always
Admissible	whenever offered and within <code>validity_distance_m</code>	only when no nominated refuge is available
Sensors	N16 free space	N16 free space + N13 2D lidar + N14 lane edge + N15 ultrasonic

The asymmetry is the whole argument

An island-detected refuge is a **physically clear lateral space** and nothing more — a verge, a hard shoulder, a wide kerb, and equally a cycle lane, a bus stop or a driveway, which the island *cannot tell apart*. Admitting one means preferring an unlawful stopping place to a stop in a live running lane.

Search, not selection

The island takes the **first** lateral clearance that passes verification, inside T_{refuge} . It weighs no candidates against each other for legality, and defers no stop to reach a better place further on — which is what keeps map and planner semantics off the island (R4, R5).

Takeaway. Three outcomes, and they are *not* interchangeable: **found** → pull over; **positively none** → in-lane stop, the corridor still in hand; **inconclusive at** T_{refuge} → **B4**, because an inconclusive search is a statement about *capability*.

Two nodes carry it — and Autoware already has the slot

#	Node	Purpose	Rate
N18	si_pull_over_manager	Implements Autoware's existing pull_over_manager MRM slot — OperateMrm service server, MrmBehaviorStatus publisher — so I7 selects it natively. Owns refuge go/no-go against N16 , longitudinal shifting within the window, the availability flag, and the abort ladder	20 Hz
N7	si_mrm_planner (extended)	Was a longitudinal ramp. Now emits a bounded autoware_planning_msgs/Trajectory (≤ 50 points) carrying both the lateral displacement path and its velocity profile. Hard bounds on lateral acceleration, steering rate and jerk applied here	50 Hz

Extending N7 to emit a standard Trajectory rather than adding a separate path planner is the cheaper move: **I13** pure_pursuit then consumes the fallback path through **exactly the same interface, unmodified**, and the fallback tier looks identical to the nominal tier from the gate's point of view.

Already upstream

mrm_handler defines /system/mrm/pull_over_manager/status, operate. **Nothing new is needed for MRM selection.** What is missing is the reporting enum: MrmState.behavior carries only NONE, EMERGENCY_STOP, COMFORTABLE_STOP.

The abort ladder

Pull-over must be abandonable at every point into a *strictly simpler* manoeuvre. This is the property that makes it safe to attempt at all.

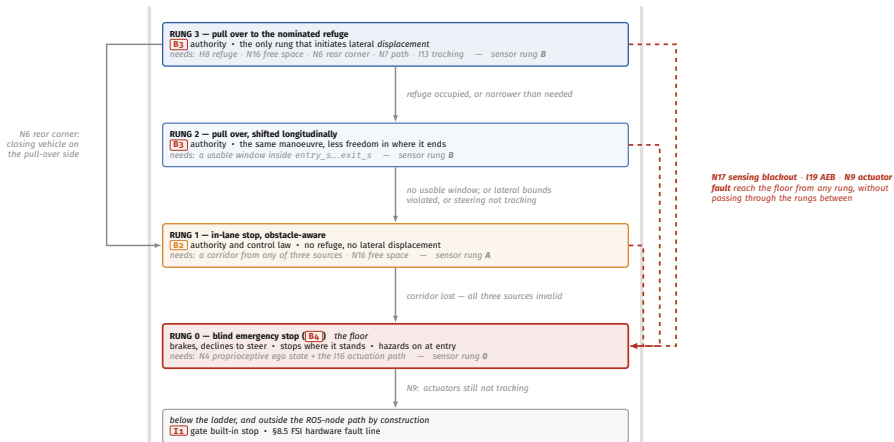
Trigger	Response
Refuge occupied or narrower than needed No usable window remains	Shift within <code>entry_s...exit_s</code> In-lane stop
Refuge search <i>concludes</i> : neither source offers one T_{refuge} elapsed, search still <i>inconclusive</i>	In-lane stop — the corridor is still there Emergency stop (B4)
Rear-corner radar detects a closing vehicle on the pull-over side Lateral path bounds violated, or steering not tracking	Hold lateral, keep decelerating → in-lane stop Freeze lateral → in-lane stop
I19 AEB fires Actuator not responding N9 Sensing blackout N17 — the refuge can no longer be verified free	Emergency stop (B4) Emergency stop (B4) Emergency stop (B4); the in-lane stop is <i>not</i> available either
Standstill reached, near field not clear	Hold, do not release; lamps stay on — raised at entry

pull-over → in-lane stop → emergency stop → gate built-in stop

Takeaway. Every rung degrades toward something the island can *definitely* do. **The manoeuvre never escalates in complexity under fault.** The two new rungs differ: a search that **concludes** still has a corridor; one that **cannot conclude** is a capability statement, and capability loss goes to the floor.

The abort ladder, drawn

new in 20260903



Takeaway. Each rung down removes a requirement and never adds one. That is what makes attempting rung 3 acceptable — the cost of failing at it is bounded by what is underneath, and what is underneath is always strictly simpler.

ODD degradation

Nothing has failed. The world
left the envelope — so constrain
the HPC before replacing it.



THE
AUTWARE
FOUNDATION

A different trigger class

	HPC failure	ODD exit — weather / visibility
Standard	ISO 26262 — something malfunctioned	ISO 21448 / SOTIF — nothing malfunctioned
HPC state	Dead, hung, or lying	Healthy and correct by its own lights
Time constant	Milliseconds	Seconds to minutes
First response	Seize authority	Constrain, and let the HPC act
Who pulls over	The island, because the HPC cannot	The HPC, because it has the map

The last row is the important inversion. When visibility collapses, the *best* outcome is that the HPC executes a planned, comfortable, legal pull-over *while it still can*. The island's job is to **demand that, then verify compliance within a deadline** — and take authority only if the HPC does not act.

Why the island must detect this independently

An island that only reads the HPC's self-reported perception confidence cannot detect the case that matters: **the HPC is confidently wrong**. No amount of health monitoring reaches it. The only thing that does is an independent sensing channel that degrades *differently*. *This is the strongest argument for the D7 sensor set — stronger than the fallback argument.*

Detection — concrete signals, no learning required

Signal	Sensor	Indicates
Max return range collapse; intensity drop; dense near-range returns	2D lidar	Fog, heavy rain, spray, snow
Multi-echo / dual-return ratio rise	2D lidar	Airborne particulates
Radar-vs-lidar detection divergence — radar reports targets lidar cannot confirm at the same range	radar + lidar	Visibility collapse, by physical diversity rather than self-report
Contrast / MTF collapse; sky-to-scene luminance; flare area; drop and blur detection	camera	Fog, glare, low sun, dirty or wet lens
Lane-detection continuity and confidence	camera	Markings obscured by snow, water film, wear
Longitudinal slip ratio vs. commanded acceleration	wheel speed + IMU	Low friction — ice, standing water
Yaw-rate vs. steering-angle model residual	IMU + steering	Reduced lateral grip
Wiper state, exterior temperature, rain sensor	vehicle CAN	Cheap corroboration

Build the radar-versus-lidar divergence test first. It is a *physical* diversity argument: radar penetrates what lidar does not, so persistent disagreement at the same range **is** reduced visibility. Nothing about it depends on a model of the weather.

Friction matters as much as visibility and is usually forgotten. It bounds achievable deceleration → stopping distance → both the speed limit and the refuge window.

The speed law

$$v_{\max} = \sqrt{2 \cdot a_{\text{brake}} \cdot (d_{\text{visible}} - d_{\text{margin}})}$$

d_{visible} measured sensing range, minimum over the active modalities

N19

a_{brake} achievable deceleration, bounded by the friction estimate

N19

d_{margin} reaction and actuation margin at the current speed

You may drive only as fast as you can stop within what you can see

Worked example — $d_{\text{visible}} = 30$ m in fog, $a_{\text{brake}} = 3$ m/s² on a wet surface, $d_{\text{margin}} = 5$ m:

$$v_{\max} = \sqrt{2 \cdot 3 \cdot 25} = 12.2 \text{ m/s} \approx 44 \text{ km/h}$$

Takeaway. Both inputs are measured on the island, so the limit is defensible without trusting the HPC. Closed-form, auditable, and appropriate for an ASIL-B node — no lookup table.

The graded degradation ladder

ODD margin	Island action	Mechanism	HPC deadline
Nominal	none	—	—
Marginal	Notify. Publish the reduced verdict; HPC re-plans conservatively	OddVerdict + mode ALLOW	—
Unsatisfiable	Demand an ODD switch. The HPC must move to a domain it can meet. The island names no target	<code>odd_satisfiable = false</code>	T_{ack} 2s declare T_{odd} 10s comply
Degraded	Restrict. Hard speed cap from the law; enlarge margins — held underneath the switch, not instead of it	mode CONSTRAIN	3s to comply
Beyond ODD, HPC compliant	Request MRM. Ask the HPC to pull over <i>while it still can</i> — and where <i>no domain is satisfiable</i> routes	mode MRM + refuge request	10s to begin 60s to complete
Beyond ODD, non-compliant	Take authority — island pull-over	I7 → N18	—

Compliance supervision

The island issues a constraint, then watches whether actual speed converges within the deadline. **Non-compliance is itself a fault** — and one the island detects *without any HPC cooperation*, from vehicle status it already receives.

Asymmetric hysteresis

Weather flickers. **N19** degrades on ~2s of evidence and recovers only after ~30s of clear evidence.
Degrade fast, recover slowly — and recovery additionally requires that the HPC's own verdict agrees.

Two nodes, two questions — and where the features meet

Node	Question	Answer, and where it escalates
N17 si_sensor_health	“Is my sensor working?” A <i>device</i> question — blockage, dirt, frame rate, internal fault	Per-source validity flag → the fault manager . A blinded sensor is a <i>fault</i>
N19 si_odd_monitor	“Is the world outside my envelope?” An <i>environment</i> question	ODD verdict with a margin → the degradation ladder . Reduced visibility is <i>not</i> a fault

Verdict arbitration

- ▶ **The more restrictive verdict binds.** Either domain may restrict; neither may relax the other.
- ▶ **Disagreement is itself a signal.** HPC says in-ODD, island says out-of-ODD — precisely the confidently-wrong case. Recorded by **N11** as a SOTIF event: evidence about the HPC’s perception that no other mechanism produces.

Fallback availability is an ODD dimension

If **N18** reports pull_over_available = false — tunnel, barriered section, no shoulder — **and** **N19** reports falling visibility, the vehicle is entering a stretch where the degraded fallback is *in-lane stop on a road it cannot see*. Neither signal alone justifies much; together they justify a much tighter speed ceiling, or not entering at all.

Ten signals in, nine out,
and not one of them a hard
dependency of the safety core.

Interfaces and deployment



HPC → island: ten typed signals

Signal	Type	Rate	Age limit	Protection
Trajectory (≤ 250 pts)	autoware_planning_msgs/Trajectory	10 Hz	300 ms	SI-E2E/A
Drivable corridor (<i>preferred</i>)	SafetyCorridor	10 Hz	300 ms	SI-E2E/A
Pull-over refuge	PullOverRefuge	2 Hz	2 s	SI-E2E/A
Kinematic state (<i>reference</i>)	nav_msgs/Odometry	50 Hz	60 ms	SI-E2E/A
Acceleration (<i>reference</i>)	AccelWithCovarianceStamped	50 Hz	60 ms	SI-E2E/A
Operation mode state	OperationModeState	10 Hz	300 ms	SI-E2E/A
Capability vector	HpcCapability	10 Hz	300 ms	SI-E2E/B
ODD status	OddStatus	10 Hz	300 ms	SI-E2E/B
Heartbeat + self-test	HpcHeartbeat	100 Hz	20 ms	SI-E2E/B
Auxiliary commands	gear, turn_indicators, hazardLights	10 Hz	300 ms	SI-E2E/A

Every input is degradable. Kinematic state → **N4** dead reckoning. Corridor → **N14** perceived lanes, then a heading tube. Trajectory → the buffered last-valid, then **N7**'s generated profile. Refuge → in-lane stop. Capability and ODD → **N2** and **N17**.

Takeaway. No row on this table is a hard dependency of the safety core.

Island → HPC, and island → actuators

Signal	Type	Rate
Island safety state	IslandState — mode, fault code, authority holder	50 Hz
MRM state	autoware_adapi_v1_msgs/MrmState	10 Hz
Constraint / veto vector	SafetyEnvelopeCommand — accel, curvature, speed limits; ALLOW / CONSTRAIN / MRM	50 Hz
Executed command (echo)	autoware_control_msgs/Control	50 Hz
Vehicle status	velocity, steering, gear, control_mode	50–100 Hz
Safety world state	SafetyWorldState — nearest obstacle, TTC, free space, corridor, validity	20 Hz
Island ODD verdict	OddVerdict — visibility, precipitation, friction, derived $v_{\max}$	2–5 Hz
Fallback availability	FallbackAvailability — pull_over_available, reason, distance	2 Hz

Island → actuator gateway

CAN FD or deterministic Ethernet at **100 Hz**, protection **SI-E2E/B**.
Actuator feedback returns on the same bus for **N9**. **This path shares nothing with the HPC link.**

On-SoC FSI → island

SPI plus a discrete SOC_ERROR GPIO — hardware faults, watchdog expiry, failover triggers — **bypassing Linux and the TSN switch entirely**.
Sub-millisecond detection; the 20 ms heartbeat is the *vehicle-level-independent* path.

In-house safety-message protection

D6 originally named AUTOSAR E2E. The WG has scoped this design as **Autoware in-house**, so AUTOSAR is out. **The obligation is not:** it comes from the communication threat analysis in EN 50159 and ISO 26262-6, and it does not care which implementation carries it.

What the receiver must still detect

Threat	Detected by
Corruption	CRC
Unintended repetition	counter
Incorrect sequence, reordering	counter
Loss	counter gap
Unacceptable delay, staleness	timestamp vs. age limit
Insertion, masquerade	Data ID
Addressing error	Data ID
Truncation, malformed payload	length

Two profiles cover the whole interface

	SI-E2E/A	SI-E2E/B
Used for	Ethernet, variable length	compact status, CAN FD
Data ID	16-bit	8-bit
Counter	8-bit	4-bit
Length	16-bit	implicit
Timestamp	64-bit gPTP	32-bit ms
CRC	CRC-32	CRC-8
Overhead	17 bytes	6 bytes

The rule that actually matters

N1 verifies on ingress, **H1** applies on egress, and a **verification failure is a safety event raised to N8** — never a middleware exception. That is independent of which profile carries it.

nano-ros already lists an E2E protocol with CRC and an EN 50159 mapping. **Part 3's first task is to check whether it covers SI-E2E/A and /B outright** — if so the island inherits it, along with the middleware's own verification evidence, instead of implementing a protocol.

Execution partitions — priority encodes the degradation strategy

Partition	Nodes	Period	Priority
P0 Safety core	I1, I2, I3, I5, I6, I7, I16, N1, N2, N3, N4, N8	10 ms (100 Hz)	1
P1 Supervision + fallback	I8, I13, N5, N7, N9, N10, N12, N18	20 ms (50 Hz)	2
P2 Safety sensing	I19, I20, N6, N13, N14, N15, N16, N17, N19	20–50 ms	3
P3 Nominal control	I9, I10, I11, I12, I14, I15	20–30 ms	4
P4 Non-real-time	N11, telemetry egress	100–1000 ms	5

Why the QM nominal controller sits *below* ASIL-B sensing Under overload the island sheds the expensive MPC first — the cheap ASIL-B fallback controller in P1 takes over within one cycle, which is exactly what the two-tier design exists to make possible. Shedding sensing instead would silently downgrade an obstacle-aware MRM to a blind one — the worse outcome.

Takeaway. P0 must complete without P2 or P3. Already exhibited by the measured failover (9.96 ms mean through a Linux outage); Part 2 must *prove* it rather than observe it.

Fault-reaction budget, and the nano-ros realisation

Preliminary budget

Heartbeat / deadline miss N2	≤ 30 ms
FSI hardware fault line	≤ 1 ms
Decision N8 $\rightarrow$ N3 $\rightarrow$ I2/I3	≤ 20 ms
Command I1 $\rightarrow$ I16 $\rightarrow$ CAN FD	≤ 20 ms
Island reaction total	≤ 70 ms
Actuator response	vehicle-specific

The 500 ms watchdog in the evaluation island is a coarse outer bound; 30 ms heartbeat detection is the production target, with the watchdog kept as the independent backstop.

What nano-ros buys the allocation

- ▶ no_std **Rust** — D5's "memory safe" at the *language* level
- ▶ **148 Kani** BMC harnesses + **83 Verus** proofs over CDR serialization, scheduling, protocol correctness — reusable ISO 26262 evidence for the hardest element
- ▶ Cyclone DDS backend **wire-compatible with stock ROS 2 RTPS** — no bridge, no type adaptation
- ▶ RMW, domain and topology are **configuration, not code**
- ▶ Verified on Zephyr, FreeRTOS, NuttX, ThreadX, bare-metal

The allocation-policy decision

XRCE-DDS allocates *only at entity setup*; zenoh-pico needs a bump arena; Cyclone DDS allocates *per message*. `rmw-alloc-sites.py` makes the bound **auditable rather than asserted**.

Minimal configuration

The smallest island that
can still pull the vehicle
over: 21 nodes, four sensors.



THE
AUTWARE
FOUNDATION

Minimal sensor set for a pull-over MRM

A ladder, not a single answer — it depends on the declared ODD.

Rung	Sensors	Sufficient for
0 the floor	island IMU / wheel speed / steering report — no exteroceptive sensor	B4 and nothing else: a blind stop in the running lane. Not a configuration anyone deploys — it is what A–C degrade to
A irreducible	2D lidar ($\geq 180^\circ$) + island IMU / wheel speed	Fair weather, low speed, road with a continuous physical edge. Where the NTU golf-cart prototype starts
B design point	+ forward radar, rear-corner radar (pull-over side)	A credible L4 ODD: weather robustness, closing speed, awareness of the lateral space crossed
C full	+ forward camera, ultrasonic array	Roads without a continuous edge; tight-to-kerb termination; and ODD degradation

Takeaway. Answer the WG's two questions together and the camera stops being optional — visibility estimation needs both the radar/lidar divergence test and the camera contrast test.

Minimal sensor set — why each one

Sensor	Verdict	Why
2D lidar	cannot be dropped	Pull-over is a question about <i>space</i> . Lidar measures free space and the physical edge directly; radar's azimuth resolution and stationary-target rejection make it a poor substitute
IMU + wheel	mandatory	No N4 , no ego state after HPC loss, nothing to track a path against
Rear-corner radar	mandatory >30 km/h	The manoeuvre crosses lateral traffic space — the gap the earlier revision did not cover
Forward radar	mandatory with weather	The only forward channel surviving fog, rain, darkness — exactly when MRM is needed
Forward camera	conditional → mandatory	Droppable for pull-over alone; not once ODD degradation is in scope
Ultrasonic	conditional	Needed for tight-to-kerb termination or standstill-release safety

Takeaway. Pull-over alone permits rung **B**. Pull-over *plus* ODD degradation requires rung **C** — **the camera stops being optional**.

Minimal node set — 21 mandatory

Group	Nodes	#
Authority and state	I1 gate · I2 decider · I3 switcher · I7 mrm_handler · N2 supervisor · N3 state machine · N8 fault manager	7
Ingress, ego state, actuation	N1 gateway · N4 vehicle state estimator · I16 vehicle interface	3
MRM behaviours	N18 pull_over_manager · I6 emergency stop (<i>abort</i>) · I5 stop_mode_operator (<i>terminal</i> + <i>auto-park</i>)	3
Fallback control	N7 mrm_planner (<i>bounded Trajectory</i>) · I13 pure_pursuit	2
Sensing	N13 scan · N6 radar · N16 world model · N17 health · I19 AEB	5
Execution assurance	N9 actuator supervisor	1

Recommended (+3): **I9** + **I12** + **I11**. The MRM needs a longitudinal law — reusing Autoware's PID beats writing one, for the same provenance reason the whole design rests on.

The two-tier argument, concluded

I10 mpc_lateral_controller is **not** in this set. **The pull-over MRM does not need the MPC. 21 mandatory, 24 as recommended — against 37 in the full design.**

What is deliberately excluded, and the risk accepted

A minimal set is only useful if the omissions are *argued* rather than overlooked.

Excluded	Why safe to omit for MRM execution	Risk accepted
I10 MPC lateral	Nominal comfort controller; the MRM tracks a bounded path with pure pursuit	Ride quality. Irrelevant
I8 validator, N10 envelope	N7 applies hard lateral-accel, steering-rate and jerk bounds at generation time	Loss of the guardian role — can still fall back, cannot veto a <i>healthy</i> HPC. Restores in phase 2
N5 corridor monitor	Pull-over deliberately leaves the lane; departure checking must be suppressed anyway	No lateral containment check during nominal driving
N12 time quality	After HPC loss the gTP master may be gone; age checks become local	Cross-domain freshness claims unavailable — and by then there is no cross-domain traffic
N11 event recorder	Not needed to <i>execute</i>	Audit (step 4) unmet. Required before a pilot, not before a bench demo
I14 shift decider	I5 's enable_auto_parking covers gear P	none
I15 raw cmd converter	Only for pedal/torque interfaces. Not needed on the NTU golf cart (steering angle + velocity)	Vehicle-specific; re-add per platform
I20 , N15	Rungs A and B have no ultrasonic	No near-field guard for the final metres. Pair with a “stop within the shoulder” terminal condition
N14 vision guard	Rungs A and B have no camera	Only valid where a continuous physical edge exists — invalid once ODD degradation is in scope
N19 ODD monitor	Not required to execute an MRM	The feature is simply absent

Thirteen questions for the WG,
and the shortest credible path
to a demonstrable fallback.

Decisions and roadmap



Open decisions for the WG — 1 of 4

1. **Target or legacy command architecture?** Recommendation: the target set (control_command_gate + command_mode_decider/switcher + stop_mode_operator), pinned to a specific Autoware release — the existing port targets Humble-era interfaces.
2. **Is a QM-rated MPC under an ASIL-D gate acceptable?** Determines whether autoware_pure_pursuit is in scope for phase 1.
3. **Which sensors, exactly?** D7 settles the principle; the concrete list and mounting remain. Each sensor costs P2 budget and a safety manual.
4. **Corridor preference order.** Map corridor first, perceived lanes second, heading tube last — confirm, and confirm the age at which each source expires.
5. **Message set ownership.** Ten new types. Proposal: one autoware_safety_island_msgs package contributed upstream, so both domains generate from the same .msg sources.
6. **In-house protection profiles.** AUTOSAR is out of scope by WG decision, so SI-E2E/A and SI-E2E/B are proposed instead. Confirm field widths and CRC polynomials — and first, whether nano-ros's built-in E2E already covers them.
7. **One RMW backend on the island, or two?** A system.toml decision, not a code decision.

Open decisions for the WG — 2 of 4

8. **Classical CV, or a certified smart camera?** (b) *recommended* — it collapses N14 to a bus parser, removes learning-based code from the island entirely, and buys supplier ASIL evidence. It costs BOM and vendor lock-in. If (a), the island's ODD must narrow.
9. **Do island sensors also feed the HPC?** The design says no — one-way, with an advisory `SafetyWorldState` upward. The alternative quietly creates the reverse dependency the independence argument forbids.
10. **MrmState has no pull-over value.** `mrm_handler` already defines a `pull_over_manager` slot, but the AD API enum carries only `NONE`, `EMERGENCY_STOP`, `COMFORTABLE_STOP`. Proposal: contribute `PULL_OVER` upstream rather than overloading `COMFORTABLE_STOP`.
11. **Can the HPC honour the refuge coverage guarantee?** A real constraint on the planner — confirm with the planning maintainers before the interface is fixed.
12. **Which ODD verdict binds?** Proposal: the more restrictive, with disagreement recorded as a SOTIF event. HPC-wins removes the only mechanism that catches a confidently-wrong HPC.
13. **What is the island's own ODD?** Narrower than the vehicle's, and it must be a first-class requirement — not discovered during validation.

Open decisions for the WG — 3 of 4: the behaviour set

14. **B2's hold window and hand-back criteria.** T_{hold} , v_{hold} , the recovery confirmation window — and the policy question of whether a recovered HPC may be trusted at all within the same drive cycle. **Blocking.**
15. **B4's blackout declaration and stop rate.** *New in version 20260903.* The debounce before N17 declares blackout — too short and a tunnel mouth stops the vehicle on a live carriageway, too long and it travels blind — plus a_{emg} , the steering-relaxation rate, and whether a blind stop must also *alert* (remote assistance, V2X, horn).

17. **The ODD switch exchange (B1).** $T_{\text{ack}} = 2 \text{ s}$ and $T_{\text{odd}} = 10 \text{ s}$ are starting proposals only — T_{odd} is additionally bounded by $t_{\text{margin}} - t_{\text{pullover}}$, so what the WG fixes is a **ceiling and a default**, not one number. **Who owns the ODD catalogue:** the design says the HPC, and the island never names a target — which keeps planning off the island, but means it cannot tell *no satisfiable domain exists from the HPC failed to find one*. **What active_odd's bounds must contain:** any bound the island cannot measure is one it cannot verify. And the policy question: **may the vehicle switch domains more than once in a drive cycle?**
18. **The island-detected refuge and T_{refuge} (B3).** The first time the island produces a stopping *place* rather than verifying one. $T_{\text{refuge}} = 5 \text{ s}$ plus a distance-searching cap; the **geometric clearance criteria**, since the island cannot classify a surface. The real question: **is an EMERGENCY_ONLY clearance acceptable at all?** The island cannot tell a hard shoulder from a cycle lane or a driveway, so admitting it means preferring an *unlawful* stopping place to a stop in a live running lane. The design says yes — second in priority, never displacing a nominated refuge. And **inconclusive versus none** must be implementable: **N18** has to be able to say which of the two it reached.

Roadmap

Phase	Adds	Capability reached
0 <i>done</i>	I9–I12	Trajectory following on the island, closed loop against unmodified Autoware
1	P0: I1–I3, I5–I7, I16, N1–N4, N8	Autonomous fallback. Authority transfer on HPC loss; blind but bounded MRM — this is B4
2	P1: I8, I13, N5, N7, N9, N10, N12	Guardian role. Command validation, safety envelope, veto, two-tier control
3	P2: I19, I20, N6, N13–N17 + sensor harness	Obstacle-aware MRM. The island stops for what it can see
4	N18 + H8	Pull-over MRM. The island moves the vehicle out of the running lane
5	N19	ODD degradation. Independent visibility and friction verdict; graded restriction of a healthy HPC

Shortest credible path

Phases 1, 3 and 4 reduced to their mandatory nodes deliver a pull-over-capable island in 21 nodes, deferring the phase-2 guardian role — the sequence recommended for the golf-cart prototype.

Next parts: 2 — timing and FTTI, RT contracts. 3 — message definitions, E2E configuration, island state machine. 4 — safety case, HARA, ASIL decomposition, fault-injection matrix.

Discussion

Full design document, diagrams and node tables: `L4SafetyIsland_SoftwareDesign.md`